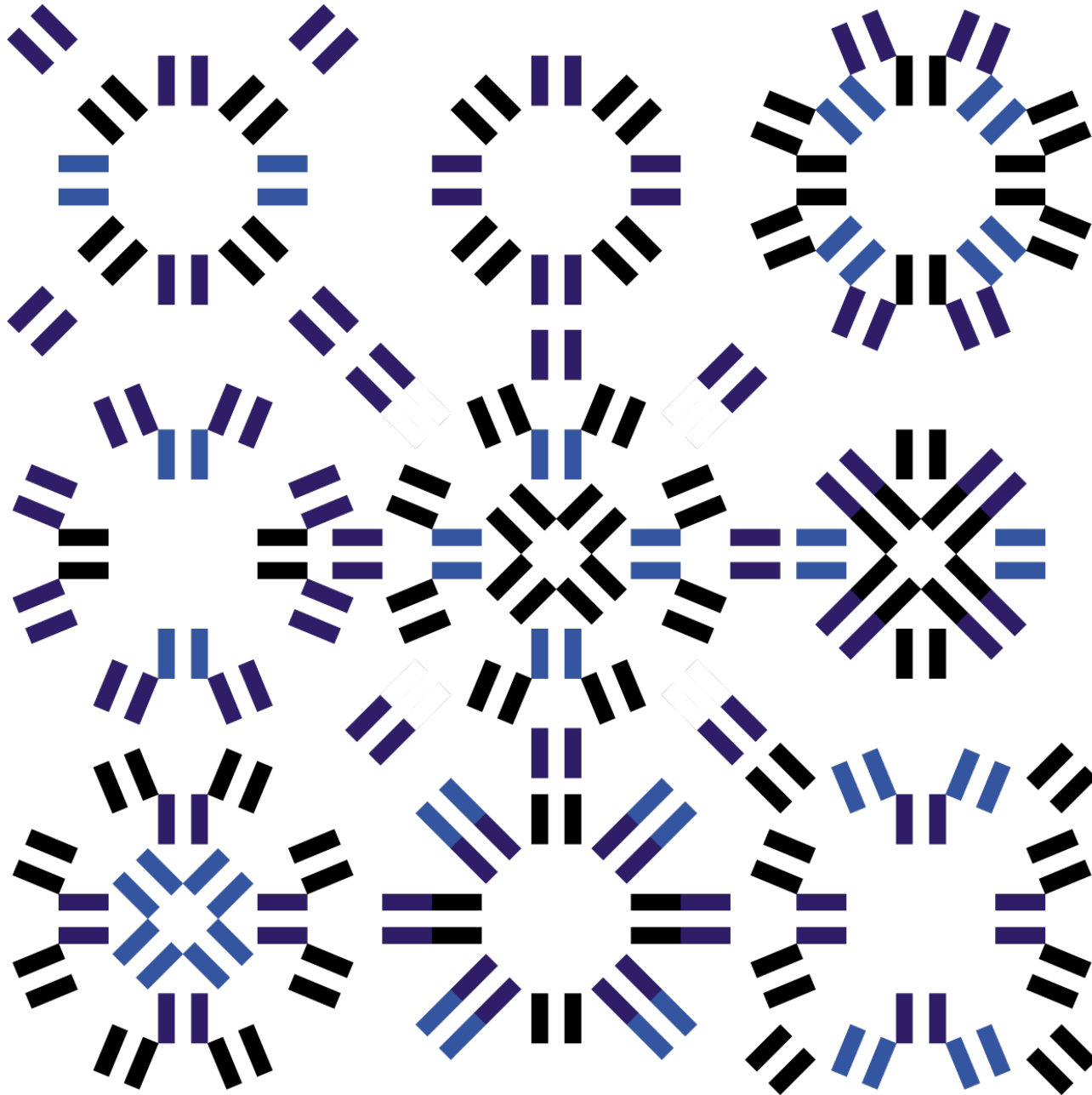




Big Data Redis

Laurent D'ORAZIO
2025-2026

Introduction

- Definition
 - Open source in-memory data structure server
 - Key-value
 - Even if value is not necessarily simple string
- Motivation
 - High performances (NoSQL)
- Use cases
 - Caching (web application)

Outline

- Data types
- Using Redis

Outline

- [Data types](#)
 - [Strings](#)
 - Lists
 - Sets
 - Hashes
 - Others
- [Using Redis](#)

String Definition

- Definition
 - Most basic type of Redis value
 - Any kind of data: JPEG image
- Limits
 - By default maximum of 512 MB

String

Basic commands – get and set – SET

- Objective

- Stores a string value

- Syntax

SET <key> <value>

- Example: store 'value' with 'mykey'

SET mykey value

String

Basic commands – get and set – SETNX

- Objective

- Stores a string value only if the key does not already exist

- Syntax

SETNX <key> <value>

- Example: store if the key does not exist 'value2' with 'mykey' (and then with 'mykey2')

SETNX mykey value2

SETNX mykey2 value2

String

Basic commands – get and set – GET

- Objective

- Retrieves a string value

- Syntax

GET <key>

- Example: retrieve the string with the key 'value'

GET mykey

String

Basic commands – get and set – MGET

- Objective

- Retrieves multiple string values in a single operation

- Syntax

MGET <key1> <key2> ... <keyn>

- Example: retrieve the strings with the key 'mykey' and 'mykey2'

MGET mykey mykey2

String

Basic commands – managing counters – INCR

- Objective
 - Increment counter

- Syntax

INCR <key>

- Example: increment 'mycnt'

INCR mycnt

String

Basic commands – managing counters – INCRBY

- Objective

- Increment counter by a value

- Syntax

```
INCRBY <key> <value>
```

- Example: increment 'mycnt' by 2

```
INCRBY mycnt 2
```

String

Basic commands – managing counters – DECR

- Objective
 - Decrement counter

- Syntax

DECR <key>

- Example: decrement 'mycnt'

DECR mycnt

Basic commands – managing counters – DECRBY

- Objective

- Decrement counter by a value

- Syntax

```
DECRBY <key> <value>
```

- Example: decrement 'mycnt' by 2

```
DECRBY mycnt 2
```

Outline

- [Data types](#)
 - Strings
 - [Lists](#)
 - Sets
 - Hashes
 - Others
- [Using Redis](#)

List Definition

- Definition
 - Linked lists of string values
- Limits
 - The max length of a Redis list is $2^{32} - 1$ (4,294,967,295) elements.

List

Basic commands – LPUSH

- Objective

- Add a new element to the head of a list

- Syntax

LPUSH <key> <value>

- Example: add element 101 to the head of work:queue:ids

LPUSH work:queue:ids 101

List

Basic commands – RPUSH

- Objective

- Add a new element to the tail of a list

- Syntax

`RPUSH <key> <value>`

- Example: add element 102 to the tail of work:queue:ids

`RPUSH work:queue:ids 102`

List

Basic commands – LLEN

- Objective

- Check the length of a list

- Syntax

LLEN <key>

- Example: check the length of work:queue:ids

LLEN work:queue:ids

List

Basic commands – LPOP

- Objective
 - Remove and return an element from the head of a list

- Syntax

LPOP <key>

- Example: check the length of work:queue:ids

LPOP work:queue:ids

List

Basic commands – RPOP

- Objective

- Remove and return an element from the tail of a list

- Syntax

RPOP <key>

- Example: check the length of work:queue:ids

RPOP work:queue:ids

List

Basic commands – LMOVE

- Objective

- Move elements from one list to another.

- Syntax

LMOVE <source> <destination> <RIGHT|LEFT> <LEFT|RIGHT>

- Example: Move elements from work:queue:ids (from the head) to work:queue:ids2 (to the head)

LMOVE work:queue:ids work:queue:ids2 LEFT LEFT

List

Basic commands – LTRIM

- Objective

- Reduce a list to the specified range of elements

- Syntax

LTRIM <key> <start> <end>

- Example: reduce the list work:queue:ids to elements 1 and 2

LTRIM work:queue:ids 1 2

List

Blocking commands – BLPOP

- Objective

- Removes and returns an element from the head of a list.
- If the list is empty, the command blocks until an element becomes available or until the specified timeout is reached.

- Syntax

BLPOP <key1> [<key2> ... <keyn>] <timeout>

- Example: Remove an element from the head of work:queue:ids with a timeout 1

BLPOP work:queue:ids 1

Blocking commands – BLMOVE

- Objective

- Atomically moves elements from a source list to a target list.
- If the source list is empty, the command will block until a new element becomes available.

- Syntax

```
BLMOVE source destination <LEFT | RIGHT> <LEFT | RIGHT> timeout
```

- Example: Move elements from work:queue:ids (from the head) to work:queue:ids2 (to the head)

```
BLMOVE work:queue:ids work:queue:ids2 LEFT LEFT 1
```

Outline

- [Data types](#)
 - Strings
 - Lists
 - [Sets](#)
 - Hashes
 - Others
- [Using Redis](#)

Sets

Definition

- Definition
 - Unordered collection of unique strings.
- Use cases
 - Track unique items
 - Represent relations
 - Perform common set operations such as intersection, unions, and differences
- Limits
 - The max size of a Redis set is $2^{32} - 1$ (4,294,967,295) members

Sets

Basic commands – SADD

- Objective

- Adds a new member to a set

- Syntax

SADD <key> <member>

- Example: add “Hello” in myset

SADD myset "Hello"

Sets

Basic commands – SREM

- Objective

- Removes the specified member from the set.

- Syntax

```
SREM <key> <member>
```

- Example: remove “Hello” in myset

```
SREM myset "Hello"
```

Sets

Basic commands — SISMEMBER

- Objective

- Tests a string for set membership.

- Syntax

```
SISMEMBER <key> <member>
```

- Example: test if “one” in myset

```
SISMEMBER myset "one"
```

```
SISMEMBER myset "two"
```

Sets

Basic commands – SCARD

- Objective

- Returns the size/cardinality of a set.

- Syntax

SCARD <key>

- Example: Size of myset

SCARD myset

Sets

Basic commands – SINTER

- Objective

- Returns the set of members that two or more sets have in common.

- Syntax

`SINTER <key1> <key2> [<key3> ... <keyn>]`

- Example: Intersection of myset and myset1

`SINTER myset myset1`

Outline

- [Data types](#)
 - Strings
 - Lists
 - Sets
 - [Hashes](#)
 - Others
- [Using Redis](#)

Hashes

Definition

- Definition
 - Record types structured as collections of field-value pairs.
- Use cases
 - Basic objects
 - Groupings of counters
- Limits
 - 4,294,967,295 ($2^{32} - 1$) field-value pairs

Hashes

Basic commands — HSET

- Objective

- Sets the value of one or more fields on a hash

- Syntax

```
HSET key field value [field value ...]
```

- Example: add user:123 with username martina, first name Martina last name Elisa country GB and age 18

```
HSET user:123 username martina firstName Martina lastName Elisa country GB age 18
```

Hashes

Basic commands – HGET

- Objective

- Returns the value at a given field

- Syntax

HGET key field

- Example: return user:123 username

HGET user:123 username

Hashes

Basic commands – HMGET

- Objective

- Returns the values at one or more given fields

- Syntax

```
HMGET key field [field ...]
```

- Example: return user:123 username and first name

```
HMGET user:123 username firstName
```

Basic commands – HINCRBY

- Objective
 - Increments the value at a given field by the integer provided.

- Syntax

`HINCRBY key field increment`

- Example: increment age of user:123

`HINCRBY user:123 age 1`

Outline

- [Data types](#)
 - Strings
 - Lists
 - Sets
 - Hashes
 - [Others](#)
 - ❖ Sorted sets
 - ❖ Streams
 - ❖ Geospatial
- [Using Redis](#)

Outline

- Data types
- [Using Redis](#)
 - Client-side caching
 - Pipelining
 - Keyspace notifications
 - Pub/sub
 - Transactions

Outline

- Data types
- [Using Redis](#)
 - Client-side caching
 - Pipelining
 - Keyspace notifications
 - Pub/sub
 - Transactions

Client-side caching

Concept

- Without caching

```
+-----+
| Application | ----- GET user:1234 -----> | Database |
|           | <---- username = Alice ----- |           |
+-----+                                     +-----+
```

- With client-side caching

```
+-----+
| Application | ( No chat needed ) | Database |
|           |                     |           |
+-----+
| Local cache |
| user:1234 = |
| username    |
| Alice       |
+-----+
```

Source: redis.io

Bibliography

- <https://redis.io/docs>

Questions?